

Teorie — operatori, condiții și bucla while

Tot ce am făcut până acum, pe scurt · C++ inițiere · Luca

Fișa asta e de **citit și de ținut la îndemână**, nu de rezolvat. Exercițiile vin separat. Fiecare bucată de cod de aici o poți scrie și rula ca s-o vezi funcționând.

1. Afișare și citire

Cum vorbește programul cu tine

```
cout << "text";      // scrie pe ecran ce e între ghilimele  
cout << x;           // scrie VALOAREA din variabila x  
cout << endl;        // trece pe rândul următor  
cin >> x;            // aștepta să tastezi ceva și pune valoarea în x
```

Într-un singur `cout` poți lega mai multe bucăți, în ordine:

```
cout << "x=" << x << endl;    // dacă x este 12, scrie: x=12
```

Diferența e importantă: `"x"` între ghilimele înseamnă litera x, iar `x` fără ghilimele înseamnă valoarea din variabilă.

2. Tipuri de date și variabile

Cele patru tipuri pe care le folosim

tip	ce ține	exemplu
int	număr întreg	7 · -3 · 1250
double	număr cu virgulă	12.7 · 3.14
char	un singur caracter	'a'
bool	adevărat sau fals	true · false

Declarare și atribuire

```
int x = 7;    // DECLARARE (o singura data) + prima valoare  
x = 12;       // ATRIBUIRE: acum x este 12  
x = x + 1;    // se calculeaza intai partea dreapta: 12+1, apoi x devine 13
```

O variabilă se declară **o singură dată**, dar poate primi oricâte valori. Ține minte **ultima** valoare atribuită.

`x = x + 1` nu e o ecuație — e o comandă: „ia ce e acum în x, adună 1, pune rezultatul înapoi în x”. Fără asta nu există bucle.

3. Operatori și operații

Operatorii aritmetici

+ adunare · - scădere · * înmulțire · / împărțire · % restul împărțirii

Capcana: dacă **ambele** numere sunt `int`, împărțirea taie zecimalele. Nu rotunjește — le aruncă.

```
5 / 2    -> 2        (int / int: zecimalele se pierd)
5 % 2    -> 1        (restul impartirii)
5.0 / 2  -> 2.5      (cel puțin unul e double)
```

```
int medie = (9 + 10 + 10) / 3;    // 29/3 = 9 (gresit, s-a pierdut ,66)
```

Perechea / și % — desfacerea unui număr

Împreună, cele două împart un număr în „câte intră” și „cât rămâne”. E cea mai folosită unealtă de până acum:

```
250 / 100 = 2    si    250 % 100 = 50    -> 2 lei si 50 bani
 47 / 10  = 4     si    47 % 10  = 7     -> 4 zeci si 7 unitati
```

Din `% 10` scoți **ultima cifră**, iar cu `/ 10` **arunci** ultima cifră. Ține minte perechea asta.

Prescurtări utile

```
x = x + 1;    se scrie mai scurt    x++;
x = x - 1;    se scrie mai scurt    x--;
s = s + i;    se scrie mai scurt    s += i;
```

4. Condiții — `if` / `else if` / `else`

Comparații și legături

Comparații: `<` mai mic · `<=` mai mic sau egal · `>` mai mare · `>=` mai mare sau egal · `==` egal

Atenție: `=` înseamnă „pune valoarea”, iar `==` înseamnă „este egal cu”. Sunt lucruri complet diferite.

Legături: `&&` înseamnă **și** (ambele trebuie să fie adevărate) · `||` înseamnă **sau** (măcar una)

```
if (varsta >= 7 && varsta <= 14)    // intre 7 si 14, inclusiv
if (nota == 8 || nota == 9)        // sau 8, sau 9
```

Cum se execută un lanț de condiții

```
if (varsta < 7) {  
    cout << "Bilet gratuit";  
} else if (varsta <= 17) {  
    cout << "Bilet redus";  
} else {  
    cout << "Bilet intreg";  
}
```

Ramurile se verifică **în ordine, de sus în jos**, și se execută **doar prima** care e adevărată. Restul sunt sărite, chiar dacă și ele ar fi adevărate.

De aceea `else if (varsta <= 17)` nu are nevoie de `&& varsta >= 7`: dacă am ajuns aici, înseamnă că prima condiție a fost falsă, deci vârsta e sigur cel puțin 7.

Regula: lanțul se încheie cu `else`. Fără el rămân valori pentru care programul **nu spune nimic** — și tăcerea nu se vede ca o eroare, programul pare că merge.

Un `if` singur, fără `else`, e corect doar când chiar vrei să nu se întâmple nimic în rest.

5. Bucla `while`

Ce e o buclă

O buclă repetă același bloc de cod de mai multe ori, în loc să-l scrii tu de zece ori. `while` se citește „**cât timp**”: cât timp condiția e adevărată, repetă.

```
int i = 1;                // 1. PORNIREA    – înainte de bucla  
while (i <= 5) {          // 2. CONDITIA    – se verifica INAINTE de fiecare repetare  
    cout << i << endl;    // corpul buclei  
    i++;                  // 3. PASUL      – inauntru, altfel nu se termina niciodata  
}
```

Ce se întâmplă de fapt, pas cu pas

Pentru codul de mai sus, programul face asta:

i	i <= 5 ?	ce se întâmplă
1	adevărat	scrie 1, apoi i devine 2
2	adevărat	scrie 2, apoi i devine 3
3	adevărat	scrie 3, apoi i devine 4
4	adevărat	scrie 4, apoi i devine 5
5	adevărat	scrie 5, apoi i devine 6
6	fals	bucla se oprește, programul merge mai departe

Condiția se verifică **înainte** de fiecare repetare, nu în timpul ei. De aceea 6 nu se mai afișează.

Cele două capcane

1. Bucla care nu se mai oprește. Dacă uiți `i++` înăuntru, `i` rămâne veșnic 1, condiția rămâne veșnic adevărată și programul scrie 1 la nesfârșit. Îl oprești din tastatură cu **Ctrl + C**.

```
int i = 1;
while (i <= 5) {
    cout << i << endl;    // lipseste i++ -> scrie 1 la infinit
}
```

2. Bucla care nu pornește deloc. Dacă la prima verificare condiția e deja falsă, corpul nu se execută **niciodată**. Nu e o eroare — e comportamentul normal, și de multe ori exact ce vrei.

```
int i = 1;
while (i <= 0) {           // fals din start
    cout << i << endl;    // nu se afiseaza nimic, niciodata
}
```

Acumulatorul și contorul

Două unelte care apar în aproape orice buclă. Amândouă se declară **înainte** de buclă și se modifică **înăuntru**:

```
int suma = 0;           // ACUMULATOR – aduna valori
suma = suma + i;        // sau, mai scurt: suma += i;

int cate = 0;           // CONTOR – numara de cate ori s-a intamplat ceva
cate = cate + 1;        // sau, mai scurt: cate++;
```

Capcana: dacă le declari **în** buclă, se resetează la fiecare repetare și rezultatul iese mereu greșit.

<pre>int suma = 0; int i = 1; while (i <= 5) { suma = suma + i; i++; } // suma = 15</pre>	<pre>int i = 1; while (i <= 5) { int suma = 0; // GRESIT suma = suma + i; i++; } // suma o ia de la 0 mereu</pre>
--	---

Trei lucruri de reținut din toată fișa:

1. `int / int` taie zecimalele — `29 / 3` este 9, nu 9.66.
2. Un lanț de `if` -uri se închide cu `else`, altfel programul tace pe unele valori.
3. O buclă `while` are trei părți: pornirea înainte, condiția în paranteză, pasul înăuntru. Dacă lipsește pasul, nu se mai oprește.

Ține fișa lângă tine când scrii cod. Se completează la fiecare lecție nouă.